

POO COM JAVA

Pedro Henrique Ribeiro Dias

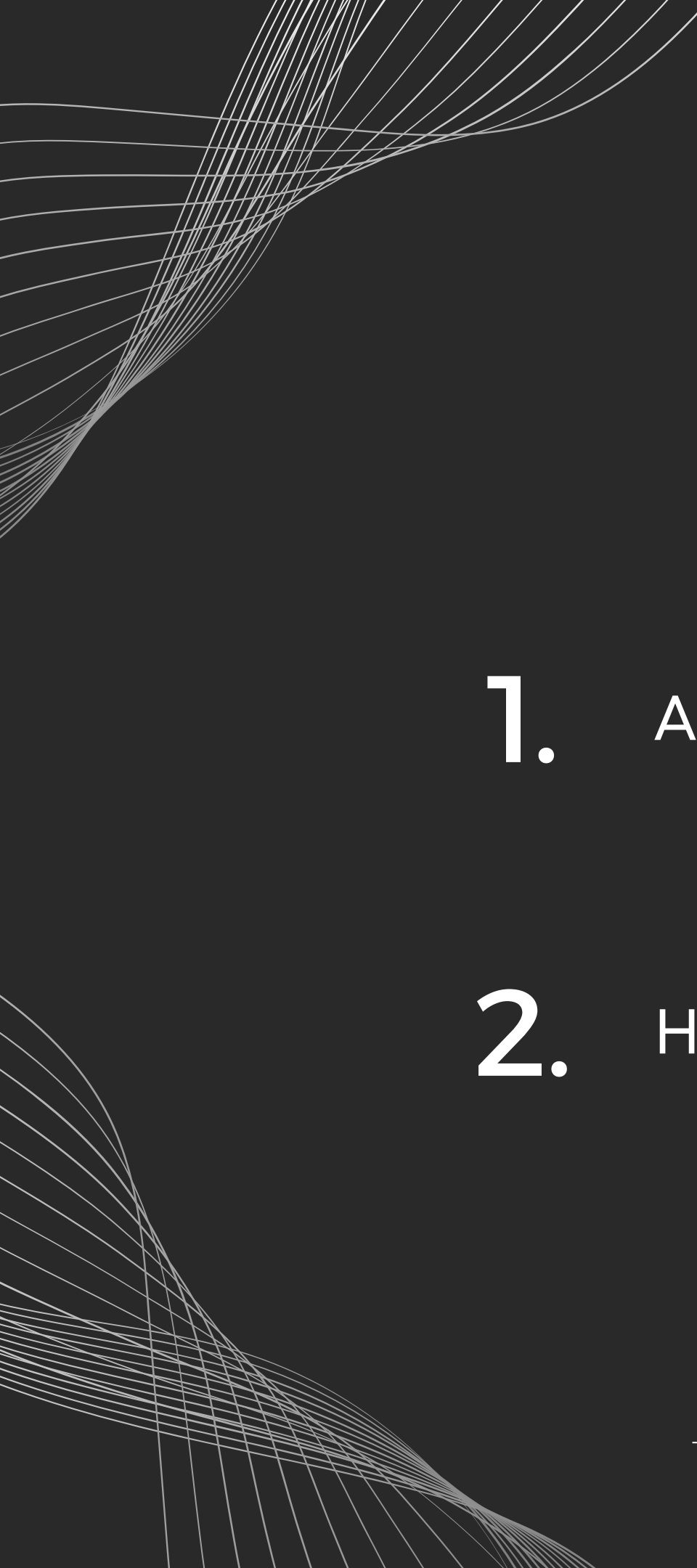
Inatel

INTRODUÇÃO

Diferente de linguagens criadas para flexibilidade ou velocidade de escrita, o Java foi construído com um objetivo claro – rodar em qualquer lugar. Lançado em 1995 com o slogan "Write Once, Run Anywhere", ele foi projetado para ser robusto, seguro e previsível, características que o tornaram a espinha dorsal de sistemas bancários, governamentais e empresariais ao redor do mundo.

O que torna o Java único?

- Portabilidade: O código Java não é compilado para uma máquina específica, mas para a JVM – a Java Virtual Machine. Isso significa que o mesmo programa roda no Windows, Linux ou Mac sem nenhuma alteração.
- Tipagem Forte: Java exige que você declare o tipo de cada variável antes de usá-la. Isso parece verboso no início, mas elimina uma classe inteira de erros em tempo de execução – o compilador pega o problema antes do programa rodar.
- Orientação a Objetos de Verdade: Diferente de linguagens onde POO é opcional, em Java tudo existe dentro de uma classe. Não há funções soltas, não há scripts avulsos – a estrutura é parte da linguagem.



PILARES DA POO

1. Abstração

2. Herança

3. Encapsulamento

4. Polimorfismo

RELACIONAMENTOS

- Composição: A classe cria e controla o objeto interno.
- Agregação: A classe recebe o objeto pronto.

Ou seja:

- ◆ Composição → dependência total
- ◆ Agregação → ligação fraca (referência)

ARRAY LIST

1. Sequência ordenada de elementos. Permite elementos duplicados. Acesso rápido a elementos por índice (.get(i)). Listas de itens onde a ordem de inserção é importante (ex: histórico de pedidos, lista de alunos por chamada).

Exemplo: `ArrayList<String> nomes = new ArrayList<>();`

HASH SET

1. O HashSet é a implementação mais performática da interface Set. Sua principal característica é não permitir elementos duplicados, garantindo a unicidade de cada item. Ele não mantém a ordem de inserção, pois a organização interna é baseada no hash code dos objetos. É a escolha ideal para coleções onde apenas a presença única do elemento importa. Exemplo de declaração: `HashSet<Integer> ids = new HashSet<>();`

HASH MAP

1. O HashMap armazena dados no formato chave-valor, sendo a implementação mais comum da interface Map. A chave deve ser única, servindo como um índice ultrarrápido para recuperar o valor associado (.get(chave)). O acesso e a manipulação são extremamente rápidos, mas a ordem de inserção não é preservada. É perfeito para mapeamentos e dicionários (ex: CPF: Pessoa).

Exemplo de declaração: `HashMap<String, Livro> catalogo = new HashMap<>();`

INTERFACES E CLASSES ABSTRATAS

- Java tem suporte nativo tanto para interfaces quanto para classes abstratas – são duas formas diferentes de definir contratos.
- A interface é criada com `interface` e define apenas o que uma classe deve ter, sem nenhuma implementação. A classe que assinar esse contrato usa `implements` e é obrigada a implementar todos os métodos.
- A classe abstrata é criada com `abstract` e fica no meio do caminho – ela pode ter métodos implementados e métodos abstratos ao mesmo tempo. A classe filha usa `extends` e é obrigada a implementar apenas os métodos marcados como `abstract`.
- A diferença prática: use interface quando classes sem nenhuma relação precisam compartilhar um contrato. Use classe abstrata quando as classes filhas compartilham uma base comum mas precisam sobrescrever partes específicas.

EXERCÍCIO 1

Cenário: O Castelo Animado do Howl está em constante movimento pelas terras de Ingary. Ele é dividido em cômodos com funções distintas, e seus habitantes têm habilidades únicas que os diferenciam de pessoas comuns.

Tarefa:

- Classe abstrata: Crie a classe abstrata Personagem com os atributos privados nome e idade. Implemente os getters para ambos e um setter para nome. Declare o método abstrato apresentar que as classes filhas serão obrigadas a implementar.
- Herança e Sobrescrita: Crie a classe Mago herdando de Personagem. O constructor deve chamar super com nome e idade. Sobrescreva o método apresentar com o comportamento específico do Mago, exibindo seu nome e alguma habilidade mágica.

○

EXERCÍCIO 1

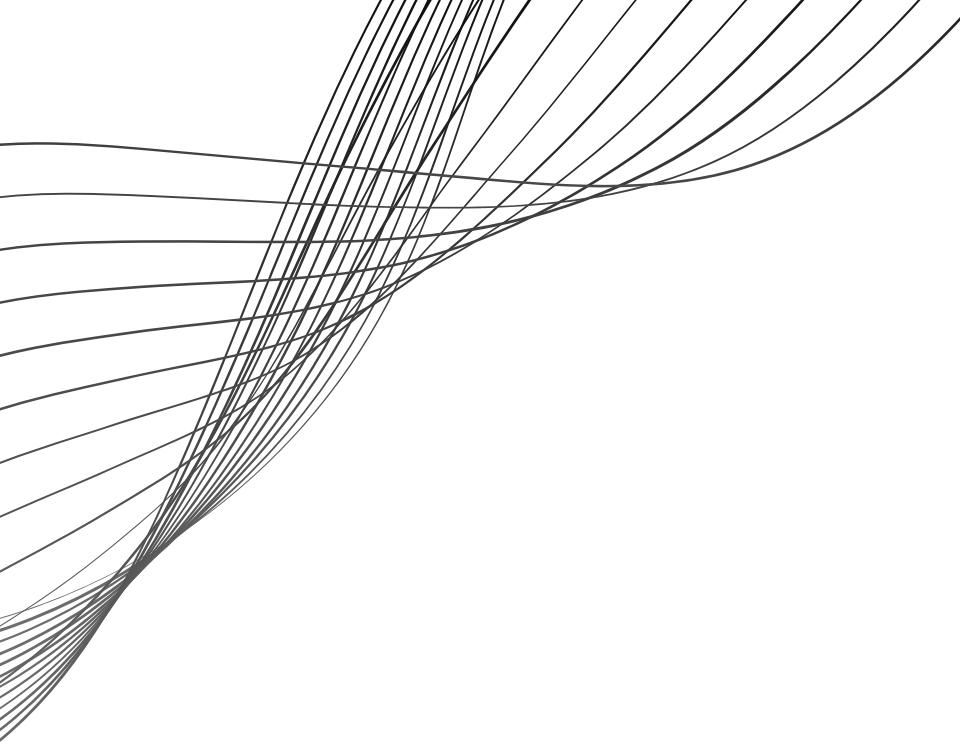
- Composição: Crie a classe Divisao com os atributos nome e funcao (ex: "Quarto", "Laboratório"). Ela não precisa herdar de ninguém – é só um molde simples.
- Agregação: Crie a classe CasteloAnimado que recebe um ArrayList<Divisao> já criado no constructor. Implemente o método adicionarDivisao para incluir uma nova divisão na lista e o método listarDivisoas que percorre o ArrayList com um for-each e exibe cada divisão.
- Main: Instancie um Mago como Howl, chame apresentar. Crie duas ou três Divisoas, agrupe-as em um CasteloAnimado e chame listarDivisoas.

EXERCÍCIO 2

Cenário: Sojiro Sakura administra o Leblanc, um café tranquilo no coração de Tokyo. O cardápio tem desde cafés simples até opções gourmet, e cada um tem sua própria forma de calcular o preço final.

Tarefa:

- Classe base: Crie a classe Cafe com os atributos privados nome e preco. Implemente os getters e o método calcularPrecoFinal que simplesmente retorna o preço.
- Herança e Sobrescrita: Crie a classe CafeGourmet herdando de Cafe. Adicione o atributo privado bonusAroma. O constructor deve chamar super com nome e preço. Sobrescreva calcularPrecoFinal com @Override somando o bonusAroma ao preço base.
- Composição: Crie a classe Menu que no constructor instancia internamente um ArrayList<Cafe>. Implemente o método adicionarCafe para incluir um café na lista e o método exibirMenu que percorre o ArrayList com um for-each exibindo o nome e o preço final de cada café via calcularPrecoFinal.



EXERCÍCIO 2

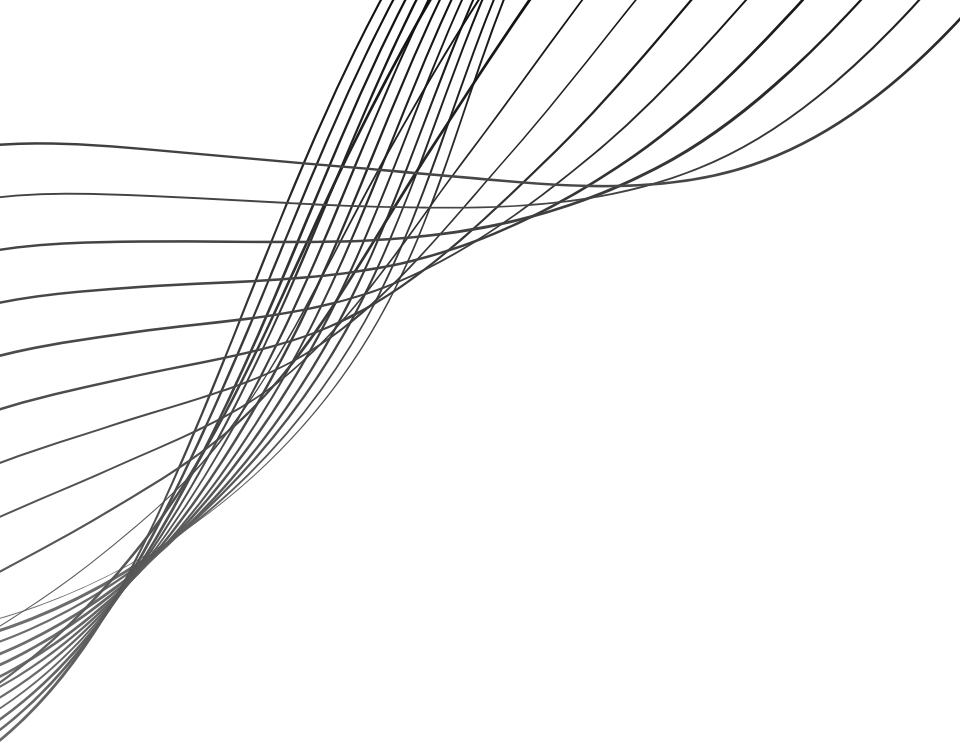
- **Composição:** Crie a classe **CafeteriaLeblanc** que no constructor instancia internamente um objeto **Menu**. Implemente o método **abrirCafeteria** que exibe uma mensagem de boas vindas e chama **exibirMenu** do menu interno.
- **Main:** Instancie dois ou três cafés, sendo pelo menos um **CafeGourmet**. Adicione-os ao menu através da **CafeteriaLeblanc** e chame **abrirCafeteria**.

EXERCÍCIO 3

Cenário: Invasões de Aliens e Youkai estão acontecendo pelo mundo e a Equipe de Detetives Ocultos precisa registrar cada avistamento. Toda entidade rastreada precisa informar suas coordenadas, e nenhuma pode ser registrada duas vezes.

Tarefa:

- Interface: Crie a interface Rastreavel com o método obterCoordenadas. Toda entidade que implementar essa interface é obrigada a ter esse método.
- Composição: Crie a classe Poder com os atributos nome e intensidade. Ela é um componente – nasce dentro da entidade e não existe sem ela.
- Herança e Interface: Crie a classe base Entidade com o atributo privado nome e seu getter. Crie as classes Alien e Youkai herdando de Entidade e implementando Rastreavel. Cada uma deve instanciar um Poder internamente no constructor e sobrescrever obterCoordenadas com um comportamento próprio.



EXERCÍCIO 3

- Agregação com HashMap: Crie a classe RegistroOculto com um `HashMap<String, Entidade>` interno. Implemente o método `registrarEntidade` que usa o nome como chave – se a entidade já existir no mapa, exibe um aviso e não adiciona. Implemente o método `listarEntidades` que percorre o HashMap com um `for-each` e exibe o nome e as coordenadas de cada entidade via `obterCoordenadas`.
- Main: Instancie dois Aliens e dois Youkais, tente registrar um deles duas vezes para demonstrar o bloqueio do HashMap, e chame `listarEntidades` ao final.

EXERCÍCIO 4

Cenário: Laios e seu grupo estão explorando as masmorras e, como sempre, Senshi transforma os monstros encontrados no caminho em refeições. Cada prato preparado tem seus ingredientes e seu método de preparo, e o grupo precisa manter um registro do que já foi cozinhado durante a expedição.

Tarefa:

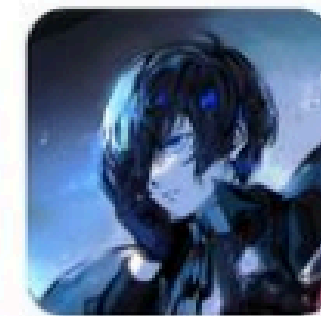
- Classe base: Crie a classe abstrata Monstro com o atributo privado nome e seu getter. Declare o método abstrato virarPrato que as classes filhas serão obrigadas a implementar.
- Herança e Sobrescrita: Crie as classes Lagosta e Cogumelo herdando de Monstro. Cada uma deve sobrescrever virarPrato exibindo uma mensagem de como Senshi prepara aquele ingrediente.
- Composição: Crie a classe Tempero com os atributos nome e quantidade. Ela é instanciada dentro de cada Prato e não existe sem ele.

EXERCÍCIO 4

- Agregação: Crie a classe Prato com o atributo nome. No constructor receba um objeto Monstro já criado e instancie um Tempero internamente. Implemente o método servir que exibe o nome do prato, chama virarPrato do monstro e exibe o tempero usado.
- Main: Instancie uma Lagosta e um Cogumelo, crie um Prato para cada um e chame servir. Em seguida crie um `ArrayList<Prato>` representando o cardápio da expedição, adicione os pratos e percorra a lista com um for-each chamando servir de cada um.

FIM DA AULA!

zSh3rl0cK/**S01-
monitoria**



 1
Contributor

 0
Issues

 0
Stars

 0
Forks



zSh3rl0cK/S01-monitoria

Contribute to zSh3rl0cK/S01-monitoria development by creating an account on GitHub.

 GitHub